



Confidential - For Kanggo Test Purpose Only

## Technical Test: Fullstack Web Developer

### Deskripsi Singkat

Buatlah aplikasi **Task Management System** sederhana yang memungkinkan pengguna untuk mengelola tugas-tugas pribadi. Aplikasi ini harus memiliki fitur autentikasi dan CRUD tugas, serta dapat dijalankan dengan stack yang telah ditentukan.

---

### Tech Stack yang Wajib Digunakan

| Bagian      | Teknologi                                                            |
|-------------|----------------------------------------------------------------------|
| Backend API | Node.js (Express.js disarankan)                                      |
| Frontend    | <b>Pilih salah satu:</b> Vue.js (Vue 3) atau React.js (dengan hooks) |
| Database    | <b>Pilih salah satu:</b> MySQL atau MongoDB                          |
| Autentikasi | JWT + bcrypt (password hashing)                                      |
| Lainnya     | Bebas (axios, tailwind, bootstrap, dll)                              |

---

### Fitur Wajib (Minimal Viable Product)

#### 1. Autentikasi Pengguna

- Register (nama, email, password)
- Login (email, password) → mengembalikan JWT token
- Logout (hapus token di sisi client)
- Proteksi route/halaman: hanya user yang sudah login dapat mengakses halaman tugas.

## 2. Manajemen Tugas (CRUD)

- Setiap tugas hanya dimiliki oleh satu user (tidak ada sharing antar user).
- Atribut tugas:
  - `id` (auto-generated)
  - `title` (string, wajib)
  - `description` (text, opsional)
  - `status` (enum: `pending`, `in-progress`, `done`)
  - `deadline` (tanggal, opsional)
  - `user_id` (relasi ke tabel/collection user)
- Fitur:
  - **Create** – form tambah tugas baru.
  - **Read** – daftar semua tugas milik user yang sedang login.
  - **Update** – edit judul, deskripsi, status, deadline.
  - **Delete** – hapus tugas (konfirmasi sederhana).

## 3. Filter Tugas berdasarkan Status

- Tersedia tombol/select untuk menampilkan tugas dengan status tertentu (`pending`, `in-progress`, `done`, atau semua).

## 4. Responsif & UX Minimal

- Tampilan tidak perlu artistik, yang penting fungsional dan rapi (gunakan CSS framework seperti Tailwind, Bootstrap, atau vanilla CSS).

---

## Fitur Tambahan (Nilai Plus – Boleh dipilih jika ada waktu)

- **Pencarian tugas** berdasarkan judul (live search atau tombol cari).
  - **Pagination** atau infinite scroll pada daftar tugas.
  - **Validasi input** di backend (misal: title tidak boleh kosong, deadline format tanggal) dan frontend (feedback error).
  - **Dokumentasi API** (Postman collection atau Swagger/OpenAPI).
  - **Unit test** sederhana (minimal 1 test untuk endpoint atau komponen).
  - **Docker support** (docker-compose untuk menjalankan app + database).
  - **Deployment** ke platform gratis (Render, Railway, Vercel + backend di service lain).  
Tidak wajib, tapi menjadi nilai tambah.
-

# Struktur dan Ekspektasi Hasil

Kandidat diminta untuk membuat **satu repository** yang berisi dua folder: **backend** dan **frontend** (atau monorepo dengan struktur jelas).

## Backend (Node.js):

- RESTful API dengan endpoint minimal:
  - `POST /api/auth/register`
  - `POST /api/auth/login`
  - `GET /api/tasks` (dengan query param `?status=...` untuk filter)
  - `POST /api/tasks`
  - `PUT /api/tasks/:id`
  - `DELETE /api/tasks/:id`
- Middleware autentikasi (verify JWT).
- Koneksi database (pilih MySQL atau MongoDB). Sertakan skema/struktur tabel atau model.
- Environment variables (`.env.example`).

## Frontend (Vue/React):

- Halaman **Login & Register**.
- Halaman utama **Daftar Tugas** (setelah login).
  - Menampilkan list tugas milik user.
  - Tombol/select untuk filter status.
  - Tombol tambah tugas (bisa modal atau halaman terpisah).
  - Setiap baris tugas memiliki tombol edit dan delete.
- Form **Tambah/Edit Tugas**.
- Logout button.
- Handle error (misal: token expired, validasi gagal).

## Database:

- Jika MySQL: buat file `schema.sql` atau migration.
  - Jika MongoDB: buat model dengan Mongoose.
-

## Deliverables

1. **Repository publik** di GitHub atau GitLab.
  2. **README.md** yang berisi:
    - Deskripsi singkat aplikasi.
    - Petunjuk menjalankan backend (termasuk setup database, env vars, install, run).
    - Petunjuk menjalankan frontend.
    - Screenshot tampilan (opsional tapi disarankan).
    - Link dokumentasi API jika ada.
  3. Rekam **video** demo aplikasi beserta penjelasan singkat dengan durasi maksimal 15 menit. Bisa diupload ke youtube (unlisted video) atau google drive.
  4. Kirim **link repository** dan **video** melalui email yang sudah ditentukan paling lambat **48 jam** sejak soal diterima.
- 

## Catatan Penting

- Dilarang menggunakan **full boilerplate** yang sudah menyediakan autentikasi + CRUD siap pakai. Diperbolehkan menggunakan library standar (express, mongoose, axios, dll).
- Jika menggunakan **React**, disarankan menggunakan `useState`, `useEffect`, dan `context` atau `redux` (boleh pakai React Query jika ingin).
- Jika menggunakan **Vue**, disarankan Composition API (Vue 3).
- Jangan lupa untuk **ignore** folder `node_modules`, `.env`, dan file sensitif lainnya di `.gitignore`.
- Pastikan aplikasi berjalan tanpa error di lingkungan lokal (prioritas utama).